Information Security

Lecture 2: The Web and Stateful Tracking

A textbook treatment

Based on the lectures, TPs and quiz of Imane Fouad

October 25, 2026

Contents

How to read this book **2**

2 The Web and Stateful Tracking **3**
2.1 The Web in Four Objects 3
2.2 Sessions and the Cookie Attachment Rule 4
2.3 Sites, Origins, Parties 5
2.4 Cross-Site Tracking versus First-Party Analytics 6
2.5 Respawning and the Other Stores 7
2.6 Cookie Syncing and Real-Time Bidding 8
2.7 Clickjacking and Framing 9

How to read this book

Each concept in this book is presented in the same rhythm, and every section closes with a *Go deeper* box (the exact details, numbers and edge cases a demanding examiner asks about) followed by *Hard questions, answered.*

1. **Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
2. **Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
3. **Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
4. **Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
5. **Examples.** From "trivially small" to "genuinely useful."
6. **Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, "say it out loud" version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Comfort with binary and hexadecimal, elementary probability, and the modular arithmetic of integers; everything else (finite fields for AES, the number theory behind RSA, Shannon entropy) is built where it is needed.

Chapter 2

The Web and Stateful Tracking

2.1 The Web in Four Objects

Definition 2.1.1 (Privacy)**.** Privacy concerns how information about people is *collected, used, shared and retained*, and a person's ability to *understand* what happens to it, *influence* its use, and *avoid* unwanted observation. A tracking mechanism "threatens privacy" when it defeats one of these verbs (invisible: understand; no opt-out: influence; survives deletion: avoid). Cases: Cambridge Analytica (Wylie, 18 March 2018); Google's $392M settlement with 40 US states (November 2022) over location tracking.

Definition 2.1.2 (URL, HTTP, HTML, browser)**.** In `https://informationsecurity.com:443/courses/web-security?week=5&topic=xss` the parts are protocol, hostname (resolved by DNS), port, path, query. HTTP is application-layer, client–server, request (URL, method GET/POST, headers, optional body) / response (status code, resource), and *stateless*: nothing is kept once the TCP connection closes. HTML is declarative structure; JavaScript is the logic. The browser loads, renders (parses HTML, runs scripts), and reacts to events (`onclick`, `onload`, `setTimeout`). `location.href` is the full URL including the fragment (never sent in HTTP), `host` includes the port, `hostname` does not, `search` is the query. History: Bush's Memex 1945, Berners-Lee's ENQUIRE 1980 (local), the 1989 proposal that became the WWW.

Key insight

Two facts drive the chapter. (1) Statelessness forces applications to add state on top of HTTP; trackers reuse that mechanism, so tracking is the web's own design pointed at another goal. (2) A script runs with the authority of the *page that includes it*, not the server that served it: `analytics.js` fetched from `metrics.test` executes as `shop.test`'s code.

Hard questions, answered

Q. `https://a.example/p#secret` includes `<script src="https://t.example/x.js">`. What does `t.example` learn from the request, and the script afterwards?
A. Request: t.example's own cookies, Referer `https://a.example/p` (no

fragment), IP, headers. The script, running as `a.example`, can read `location.href` incl. `#secret` and a.example's non-HttpOnly cookies, and send them explicitly.

2.2 Sessions and the Cookie Attachment Rule

A session token in the URL (`?sessionid=8F4A92C1`) leaks through the `Referer` header to every embedded third party, through shared links, logs and history. Cookies (1994) fix the leak and create everything else.

Definition 2.2.1 (Cookie; attachment rule)**.** A server stores a name–value pair with `Set-Cookie`; the browser returns it in `Cookie` on *every request to that domain, regardless of which site initiated the request.* Without `Expires`/`Max-Age` it is a *session cookie*, deleted when the browser closes; with one it is *persistent.*

Reading the formula

Both clauses are load-bearing. *To that domain*: no site can read or receive another's cookie, so identifiers must be smuggled through URLs to cross sites (§2.6). *Regardless of initiating site*: `bbc.com` embedding an image from `doubleclick.net` causes a request carrying doubleclick's cookie; that is third-party tracking and also CSRF. `SameSite` (1990s rule, 2016 attribute) qualifies exactly this clause. Model: a jar indexed by domain; on each request the browser attaches matching cookies without knowing which page caused the request.

Definition 2.2.2 (Attributes)**.** `Domain`: default current host only; set to a parent (`Domain=example.com` from `websec.example.com`) to cover it and all subdomains; cannot name an unrelated domain or a public suffix. `Path`: that path and sub-paths. `HttpOnly`: invisible to `document.cookie`; still *sent.* `Secure`: sent only over HTTPS and settable only from HTTPS. `SameSite=Strict`: never on cross-site requests. `Lax` (default): on cross-site requests only if *top-level navigation and* a *safe method* (GET, HEAD, OPTIONS, TRACE; RFC 7231). `None`: always, requires `Secure` (Chrome rejects None without Secure; Firefox/Edge warn).

Technique: Is cookie c attached to a request from page P to destination D?

(1) D in c's domain scope? (2) Path prefix? (3) If Secure, HTTPS? (4) Same site (registrable domain) for P and D? If yes, attach. If cross-site: Strict no; Lax only for top-level *and* GET-like; None yes. `HttpOnly` never enters: it only governs script reads.

Worked example: Quiz 1, Q4

`https://shop.example.test` sets `aid=A72; Domain=example.test; Path=/; Secure; HttpOnly; SameSite=Lax`. A: from the shop, load `https://cdn.example.test/banner.png`: subdomain cov-

ered, HTTPS, same site: *yes.* B: same over `http://`: Secure fails: *no.* C: from `https://social.test` Alice *clicks a link* to `https://shop.example.test/profile`: cross-site but top-level GET: Lax allows: *yes.* D: `social.test` *embeds* `https://shop.example.test/tracker.png`: cross-site subresource, not top-level: *no.* Answer A, C; HttpOnly was a distractor.

Gotcha

Lax needs *both* conditions: a cross-site iframe/image fails top-level; a cross-site POST form is top-level but unsafe. Only a plain link passes. HttpOnly protects against script theft (XSS), not against sending (CSRF, tracking); Secure against network theft; SameSite against cross-site attachment.

Go deeper: the details an examiner asks about

- `Set-Cookie: test=1; Domain=.example.com; Secure; HttpOnly; SameSite=None`: sent on any HTTPS request to example.com and subdomains, cross-site too, invisible to JS, and still a *session* cookie (no expiry).
- Scope is by *name*: `lab.test:8000` and `127.0.0.1:8000` are different jars for the same Flask server (TP).
- Trackers need `None; Secure`; banks prefer Strict (no CSRF at all) or Lax (usability: inbound links arrive logged in).

Hard questions, answered

Q. A bank cookie is `HttpOnly; Secure`, no SameSite. An attacker page auto-POSTs a transfer form. Sent?
A. Under the Lax default: no (unsafe method). Treated as None: yes, transfer succeeds as the victim. HttpOnly/Secure are irrelevant to attachment.

Q. Can `a.example.com` set a cookie `b.example.com` receives? One `example.org` receives?
A. Yes with `Domain=example.com`; no.

2.3 Sites, Origins, Parties

Definition 2.3.1 (eTLD+1; origin)**.** The registrable domain (**eTLD+1**) is the public suffix (`com`, `co.uk`, `github.io`) plus one label: the unit of *ownership*, read from the right. A request is **first-party** if its destination has the address bar's registrable domain, **third-party** otherwise; cookies are first/third-party accordingly. An **origin** is (scheme, host, port); the **same-origin policy** restricts what a script may *read* (documents, storage, `document.cookie`) to its own origin. Subdomains are same-site but cross-origin.

Reading the formula

Site governs *attachment* (SameSite, partitioning); origin governs *access*. The SOP never prevents the browser from *sending* a cookie (Quiz 1 Q6C is false); it prevents `bbc.com`'s script from *reading* doubleclick's cookie. Visiting `https://shop.example.com`: `www`, `images`, `api.shop`, `analytics.example.com` are first party; `example.org`, `example.com.tracker.net` (owner: `tracker.net`), `cdn.google.com`, `shop-example.com` are third party. A tracker needs three capabilities: (1) an identity in the browser (cookies, headers, storages: *stateful*; or recomputed from properties: *stateless*, Chapter 3); (2) a channel back (automatic cookie attachment, or explicit JavaScript); (3) context (Referer, `location`, a parameter).

Hard questions, answered

Q. Visiting `https://news.bbc.co.uk`: classify `static.bbc.co.uk`, `bbc.com`, `bbc.co.uk.cdn-edge.net`.
A. First; third (different registrable domain); third (owner `cdn-edge.net`).

Q. Can a script on `a.example.com` read `b.example.com`'s cookies? Are `Domain=example.com` cookies sent to both?
A. No (cross-origin); yes (same-site, Domain covers both).

2.4 Cross-Site Tracking versus First-Party Analytics

Theorem 2.4.1 (One third-party cookie)**.** *A tracker embedded on sites $\mathcal{S}$ with a persistent cookie learns, per browser, the timed sequence of visits (site and page) to every site in $\mathcal{S}$, with no cooperation from the sites.*

Proof. Each visit to $s \in \mathcal{S}$ triggers a request to the tracker ("regardless of initiating site"); it carries the tracker's cookie ("to that domain"), the same value on every site because it belongs to the tracker's domain, not to s; Referer or an embedding parameter supplies s and the page. Group the log by cookie value. □

First-party analytics looks similar and is not: `accuweather.com` includes `google-analytics.com/script.js`; the script runs as accuweather and sets `ga_id=123` *under accuweather.com*, a first-party cookie; it must send it explicitly (`track?ga_id=123&site=accuweather.com`) since the browser never attaches accuweather's cookie to a google-analytics request. On `nytimes.com` the same script creates a *different* cookie under nytimes. Google gets per-site identifiers it cannot link: within-site repeat visits only.

Key insight

The difference is *under which domain the cookie is stored*: tracker's domain = one identifier everywhere, attached automatically; publisher's domain = one identifier per publisher, sent explicitly. "Third-party cookies blocked" blocks cookies set in third-party context, not third-party *scripts* setting first-party cookies.

Worked example: Quiz 1, Q5–Q7

Q5 (non-HttpOnly first-party `prefs`; `metrics.test/analytics.js`; iframe from `tracker.test`): the script runs in the shop's origin and reads `prefs` (A), and can send it explicitly (D); the iframe is another origin (B false); the request for `analytics.js` carries metrics.test's cookies, not the shop's (C false). *Q6* (pixel from `tracker.test`, `tid; SameSite=None; Secure`, 3P allowed): same `tid` from both publishers (A), tracker links them (D); publisher JS cannot read it (B false); SOP does not govern sending (C false). *Q7* (3P blocked; same script; `aid=N31` on news, `S84` on shop): first-party identifiers (A); not sent automatically to metrics.test (B false); same script ≠ same identifier (C false); script can read and send the current one (D). Merging N31 and S84 needs an external link (login, fingerprint).

Go deeper: the details an examiner asks about

- Publishers set `HttpOnly` on their session cookie partly against the third-party scripts they include.
- Citations: Roesner et al. (NSDI 2012) classified tracking behaviours; Englehardt & Narayanan (CCS 2016, OpenWPM, 1M sites) measured third-party presence.

Hard questions, answered

Q. Minimal change turning an analytics vendor into a cross-site tracker?
A. Have the script load a resource from the vendor's domain that sets a vendor-domain cookie with `SameSite=None; Secure`; it is then attached on every site that includes the script. Under the Lax default the embedded request would not carry it.

2.5 Respawning and the Other Stores

Definition 2.5.1 (Respawning)**.** Recreating a deleted cookie from a copy held in another store (a *zombie* cookie): HTML5 `localStorage` (per origin, no expiry, across sessions), Flash local shared objects (across sessions *and* browsers, since the plugin's store was per machine user), HTTP cache headers `ETag`/`Last-Modified`. Evercookie combines many. All of these can also track *without* cookies.

Mechanics. Tracker iframe on `site1.com` writes `id=789` as cookie and into `tracker.com`'s localStorage. User deletes cookies; on `site2.com` the server issues `id=555`, the script finds 789 in localStorage and reports `old_id=555, new_id=789`; profiles merge. ETag: first response `ETag: "x234dff"`; the browser sends `If-None-Match: "x234dff"` on the next request and may get `304 Not Modified`; nothing forces the tag to describe content, so it can be a user identifier echoed even in private browsing with cookies blocked, as long as the cache lives (KissMetrics, August 2011: `_km_cid` cookie equal to the ETag). Each store has its own scope and its own clearing action; a privacy action is sufficient only if it clears every store the tracker used.

Worked example: Quiz 1, Q10–Q12

Tests: (2) delete cookie, cookie back as 1234 with localStorage 1234: *consistent with* localStorage restore (A); does not *prove* fingerprinting (B false). (3) delete cookie + localStorage (now "new"), cookie still 1234, ETag 1234: consistent with ETag restore (C); profile still 3, so deletion did not break the link (D false). (4) clear cookies, storage, cache: new 5678 everywhere (stores *were* cleared, Q11C false) but fingerprint 88 and profile 3 persist: consistent with stateless relinking (B); a fingerprint identifies a browser, not a named person (A false); shared fingerprints give false positives (D). Q12: clear cookies, local storage and cache *together* (A); the other options leave a source.

Hard questions, answered

Q. Design a respawn surviving "clear site data" but not "clear cache".
A. Identifier in the `ETag` of an always-loaded resource, echoed in `If-None-Match`; site-data clearing removes cookies and storage, not the HTTP cache.

Q. Everything cleared, profile continues. Remaining explanation?
A. A stateless identifier recomputed from browser properties: Chapter 3.

2.6 Cookie Syncing and Real-Time Bidding

Theorem 2.4.1 is bounded by $\mathcal{S}$: doubleclick sees bbc and nytimes, adnxs sees bbc and jumia, each with its own identifier, and neither can read the other's cookie.

Definition 2.6.1 (Cookie syncing)**.** The process by which two trackers link the identifiers each assigned to the same browser. Mechanism: the browser requests a pixel from tracker 1 with `Cookie: id=1234`; tracker 1 replies `302 Location: https://tracker2.com/sync?t1=1234`; the browser follows, and the request to tracker 2 carries *tracker 2's* cookie `id=5678` plus the URL parameter; tracker 2 records (5678 ↔ 1234) and may redirect back with `t2=5678`. The identifier crossed domains *inside the URL*, the one place the attachment rule does not police; neither tracker read the other's cookie.

RTB: the **SSP** offers a publisher's ad space and forwards each opportunity to the **ad exchange**, which runs the auction and picks the winner; **DSPs** evaluate and bid for advertisers. The bid request (page URL, approximate location, pseudonymous ID) is *broadcast to all DSPs*; an SSP syncs its cookie with a DSP by redirect so the DSP recognises the user in the request. Only part of the process is visible in the browser.

Worked example: Quiz 1, Q8, Q9, Q16

Q8: `GET tracker-a/sync, Cookie: aid=A42` → `302 tracker-b/sync?aid=A42` → `GET ..., Cookie: bid=B71`: B associates A42 with B71 (A); A42 is a URL parameter, not B's cookie (B false); A passed its ID in the redirected URL (C); no shared origin needed (D false). *Q9*: the SSP helps the publisher sell (A); the losing DSP-X still received the request (B); not only the winner (C false); the pseudonymous ID is stable, it enables linking (D false). *Q16* (drawing): browser in the middle with

both jars; request 1 carrying 1234; the 302 with 1234 in the URL; request 2 carrying 5678 and the parameter; the mapping table at tracker 2; optional return redirect.

Hard questions, answered

Q. Does per-site cookie partitioning (Firefox, Chapter 3) stop a sync between two trackers on the *same* publisher?
A. No: both have cookies in that site's jar and the redirect links them there. Partitioning stops reuse across publishers.

Q. Why a redirect rather than a script?
A. Only a request to tracker 2's domain carries tracker 2's cookie; the redirect produces such a request while carrying tracker 1's ID in the URL. No script can read either cookie across origins.

2.7 Clickjacking and Framing

TP1: `game.html` embeds `<iframe src="portal.html">` positioned over a "Catch Me" button, higher `z-index`, `opacity: 0`; at score 4 the button is moved under the portal's Login; Alice's click lands on the invisible, *authenticated* Login. Relies on: clicks go to the top-most element and `opacity: 0` keeps the frame in hit-testing (`visibility: hidden` or `pointer-events: none` would not); the framed page received its cookies ("regardless of initiating site"; a Lax/Strict session cookie would not arrive, which mitigates); nothing forbids framing unless the portal says so.

Definition 2.7.1 (Framing controls)**.** Response headers sent by the *framed* page. `X-Frame-Options: DENY` | `SAMEORIGIN` (`ALLOW-FROM` obsolete). CSP `frame-ancestors 'none'` | `'self'` | list of origins (`frame-ancestors 'self' https://apps.um6p.ma`); takes precedence over X-Frame-Options; the only way to whitelist a trusted third origin.

Hard questions, answered

Q. The portal must never be framed, except later by one trusted university app. Headers?
A. Now: `frame-ancestors 'none'` + `X-Frame-Options: DENY` for old browsers. Later: `frame-ancestors 'self' https://trusted.um6p.ma`; X-Frame-Options cannot express it.

Q. Setting anti-framing headers on the *game* page: useful?
A. No; the attacker controls the game page. The victim page must send them.

Summary: Stateful tracking

- Cookies are attached to every request to their domain, whoever initiated it; SameSite qualifies the second clause (Lax = top-level *and* safe method; None needs Secure).
- Site (eTLD+1) governs attachment; origin governs script access.

- One tracker-domain cookie links all embedding sites; publisher-domain analytics cookies cannot.
- Respawn from localStorage, Flash, ETag; clear all stores together; what survives everything is stateless.
- Syncing crosses domains inside a redirect URL; RTB broadcasts bid requests.
- Clickjacking: transparent authenticated iframe; defence: X-Frame-Options / CSP frame-ancestors on the framed page.